

Proyecto Aplicaciones Web 2

**COLEGIO
UNIVERSITARIO**



Integrantes:

- Colomba Matías Gabriel (43694674)
- Máximo Ezequiel Paz (44767857)
- Tano Busso (39080247)
- Bruno Ariza (46222763)

TP1: Propuesta de Trabajo

Objetivo:

Desarrollar una página web que permita al usuario seleccionar componentes de computadora y comprobar si hay alguna incompatibilidad entre ellos para reducir los problemas al momento de comprar y armar una computadora.

Descripción:

El proyecto consiste en el desarrollo de una aplicación web que permita a los usuarios verificar la compatibilidad entre distintos componentes de una computadora.

El sistema permitirá seleccionar componentes como: CPU, Motherboard, GPU, Refrigeración de CPU, Gabinete y otros componentes de computadora.

A partir de estos datos, la aplicación determinará si son compatibles entre sí.

Problemas que resuelve:

- Problemas al momento de armar la computadora.
- Incompatibilidades técnicas.
- Tiempo perdido en devoluciones o cambios.

Solución propuesta:

La aplicación permitirá validar la compatibilidad entre componentes de un sistema

El sistema analiza datos como:

- Tamaño de la GPU.
- Espacio disponible en el gabinete.
- Compatibilidad de chipset de la Motherboard y la CPU

Una vez analizado los datos, devolverá resultados como: Incompatible y Advertencia (ej: “entra justo”).

Propuesta de Valor:

- Reduce errores de compra.
- Ahorra tiempo y dinero
- Brinda una herramienta simple y útil.
- Permite escalar el sistema añadiendo más componentes y sus correspondientes validaciones.

Roles:

- Tano Busso - Front End: diseño de interfaz, estructura HTML y estilos CSS.
- Colomba Matías - Back End: desarrollo de la API y base de datos.
- Bruno - Testing: pruebas del sistema, detección de errores en la validación de componentes.
- Máximo Paz - Administrador de Repositorio: acepta Pull Request, mantiene el repositorio acomodado

Estos roles pueden ir rotando entre los integrantes a lo largo del desarrollo del proyecto.

Backend del sistema:

El proyecto no solo contempla el desarrollo de una interfaz visual, sino también la construcción de un backend que permita gestionar la lógica de negocio y los datos del sistema.

El backend será desarrollado utilizando Node.js junto con Express.js, permitiendo la creación de un servidor web capaz de:

- Servir la aplicación web al cliente
- Gestionar las solicitudes del usuario
- Procesar la lógica de compatibilidad entre componentes
- Consumir y/o exponer APIs

Funcionalidades del backend:

El backend del sistema se encargará de:

- Validar la compatibilidad entre componentes (CPU, GPU, motherboard, etc.)
- Procesar reglas de negocio (ej: compatibilidad de chipset, espacio físico)
- Obtener datos desde APIs externas (en etapas iniciales)
- Gestionar datos propios mediante una base de datos (en etapas posteriores)

Arquitectura prevista:

El sistema se desarrollará siguiendo una arquitectura basada en:

- Servidor backend (Node.js + Express)
- API REST para comunicación con el frontend
- Separación por capas (controladores, servicios, modelos)

Esto permitirá que el sistema sea:

- Escalable
- Mantenible
- Fácil de extender

Evolución del proyecto:

El desarrollo se realizará en etapas, alineadas con la materia:

- TP2: Servidor backend + consumo de API externa
- TP3: Implementación de CRUD + base de datos
- TP4: Autenticación, seguridad y gestión de usuarios

Cronograma:

Para tener registro de las fechas límite de entrega utilizamos la herramienta Trello. El siguiente es el link del cronograma:

- https://docs.google.com/spreadsheets/d/1LWX6_xM2nwUQHazp-P6tRjhPdGYJuJ8B/edit?usp=sharing&oid=102938447517335443619&rtpof=true&sd=true

Trello:

Para registrar el estado de las tareas y quien las realiza utilizamos la herramienta Trello. El siguiente es el link del Trello:

- <https://trello.com/invite/b/69ebad08e596a9275f475eb0/ATTI376df0786cd918cfaf1e3294b764f6db81FC7B75/trabajo-practico-aw2-v-20>

Herramienta de seguimiento y control de versiones:

Para este proyecto usaremos Git como herramienta y Github como plataforma. El siguiente es el link del Github:

- <https://github.com/Maximo2205/Aplicaciones-web-2---Trabajos-Prcticos>

TP2:

Back-End para servir el sitio web

En esta fase del proyecto, hemos consolidado el núcleo del sistema mediante el desarrollo de un backend en **Node.js** con el framework **Express**. El objetivo principal fue establecer un servidor funcional que actúe como intermediario para la gestión de componentes y proporcione una buena base para la lógica de compatibilidad.

1. Arquitectura del Sistema

Se ha implementado una arquitectura basada en la separación de responsabilidades:

- **Capa de Presentación (Frontend):** Interfaz desarrollada con HTML y CSS. La lógica de usuario se maneja mediante **ESM (ECMAScript Modules)**, facilitando la modularidad.
- **Capa de Servicio (Backend):** Servidor API RESTful que centraliza la lógica de negocio y la gestión de recursos.
- **Gestión de Estado:** Implementación de localStorage para la persistencia del lado del cliente, permitiendo que la selección del catálogo se mantenga al navegar hacia la vista de análisis.

2. Motor de Compatibilidad

El software incluye un motor de reglas especializado (listComp.mjs) que analiza requisitos técnicos críticos:

- **Lógica de Compatibilidad:** Cruce de datos entre sockets y chipsets de Motherboard y CPU.
- **Estándares de Memoria y Almacenamiento:** Verificación de generaciones DDR y formatos M.2.
- **Restricciones Físicas:** Comprobación de dimensiones espaciales, como la longitud de la GPU frente a la capacidad del Gabinete (Chassis).

3. Estado de Tareas (Metodología Kanban)

- **Finalizado:** Estructura de navegación, diseño UI/UX del catálogo, motor de compatibilidad inicial, servidor base y API CRUD completa.
- **En Proceso:** Integración final del frontend con el servidor local (transición de MockAPI al servidor propio).
- **Pendiente:** Implementación de Base de Datos relacional (TP3) y Sistema de Autenticación y Seguridad (TP4).

4. Conclusión y Cumplimiento de Consigna

El servidor se encuentra operativo y cumple con el requerimiento académico de servir el proyecto frontend. La estructura actual permite que la interfaz consulte y envíe datos de forma eficiente, sentando las bases para la futura migración a una base de datos persistente.

TP3:

Creación del CRUD para administrar Front-End

1. Resumen de la Etapa y Objetivos

En esta tercera fase del proyecto, el sistema **chkCompatibility** experimentó una evolución estructural crítica, transitando desde un esquema con datos aislados en memoria hacia una arquitectura robusta de producción. Los pilares alcanzados en esta entrega comprenden:

- **Migración de Arquitectura:** Reestructuración del backend adoptando el patrón de diseño **MVC (Modelo-Vista-Controlador)**.
- **Contenerización y Entorno:** Implementación de **Docker**.
- **Persistencia Avanzada:** Conexión del servidor Express a un motor de **Base de Datos** relacional de manera local (PostgreSQL).

2. Reestructuración de la Arquitectura (Patrón MVC)

El proyecto fue reestructurado al Patrón MVC visto en clases, siendo una carpeta para los módulos y dentro de esta misma carpetas individuales de cada módulo, de momento estando la carpeta “Componentes” con los archivos .mjs del modelo, controlador y rutas.

3. Implementación de MockAPI y Estrategia de Integración

Temporalmente utilizamos una API diseñada en MockAPI que funcionara como una base de datos provisional en donde pudiéramos integrar los datos mostrados en la página web. La API usada para el proyecto se encuentra en el siguiente link:

'<https://69f3f9acbd2396bf5310826e.mockapi.io/api/v1/componente>'

Actualmente descartamos la API y la reemplazamos por una base de datos diseñada e integrada en el programa por medio del sistema Docker.

4. Relación entre MockAPI y la Base de Datos Integrada en Docker

El uso de MockAPI no reemplaza a la base de datos persistente, sino que actúa como su **esquema espejo de preproducción**. La coexistencia e integración de ambas tecnologías se define bajo los siguientes lineamientos técnicos:

- **Homologación de Modelos:** El JSON estructurado en MockAPI (que incluye campos críticos como name, tipo, cpuSocket, ramDdr, entre otros) fue el plano de diseño utilizado para estructurar las tablas y entidades dentro de la base de datos relacional montada bajo Docker.
- **Flujo de Migración de Datos:** El backend desarrollado bajo arquitectura MVC ya cuenta con los endpoints (GET)
- **Estrategia de Desacoplamiento (Swapping):** Al haber estructurado las funciones del frontend utilizando variables dinámicas (fetch(url)), el paso definitivo de producción consiste únicamente en modificar la constante url en los archivos .mjs para que apunten al entorno local dockerizado (<http://localhost:3000/api/v1/componentes>), garantizando que la aplicación siga funcionando idénticamente pero con persistencia real y local.

5. Entorno de Desarrollo y Contenerización con Docker

Para asegurar la portabilidad y homogeneidad del sistema entre todos los integrantes del equipo, la infraestructura local se orquestó mediante contenedores independientes:

1. **Contenedor del Servidor (Express):** Ejecuta la lógica MVC en Node.js, sirviendo tanto los archivos estáticos del frontend en el puerto 3000 como respondiendo a las llamadas de la API interna.



2. **Contenedor de Base de Datos:** Instancia aislada del motor relacional que gestiona la persistencia real del catálogo de hardware.